# Mastering Augmented Reality with Unity

Prof. Jooho Cha, Ph.D.
jhcha@chungwoon.ac.kr

Dept. of Multimedia Science
Chungwoon University

# Chapter 7. Trying Out First 3D Mobile App Development

- Introduction
  - In AR application development, one of the major steps is to iteratively test the application on real devices to ensure usability and optimization.
  - We are going to go over the steps to build a mobile application on Android and iOS devices.
  - We will also be going through the essential tools, and describing the debugging steps required to check the status of the app while it's running on mobile devices.

# Chapter 7. Trying Out First 3D Mobile App Development

- Structure
  - Unity build settings: Android & iOS build settings
  - Building unity apps: Building for Android & iOS
  - Unity script debugging with Visual Studio
  - Debugging the app for both platforms



# Chapter 7. Trying Out First 3D Mobile App Development

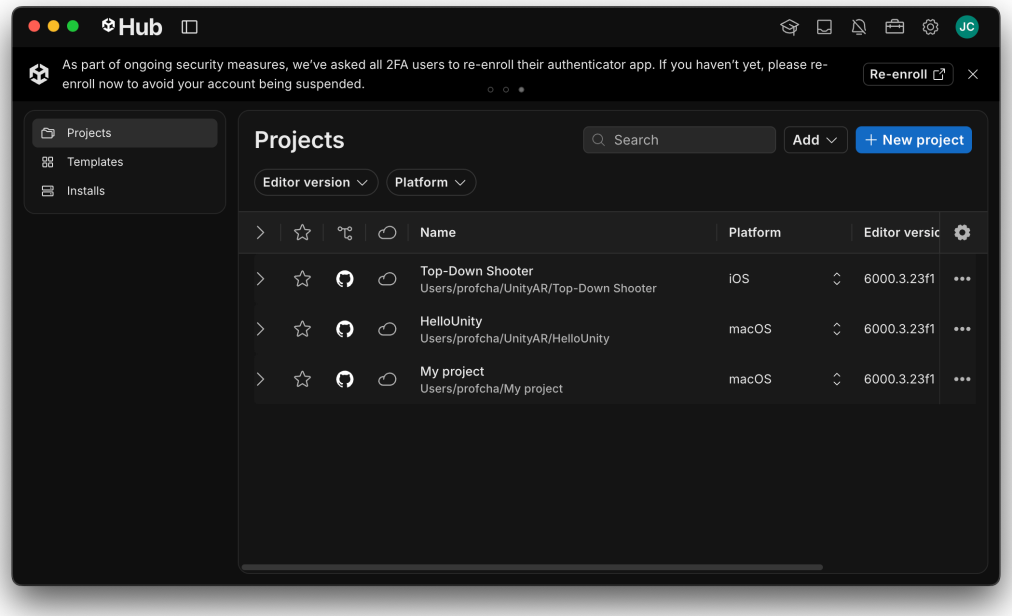
- Objectives
  - You will be able to optimize and build mobile applications that run on Android and iOS operating systems.
  - You will also be able to connect your device to the computer to debug and test your code.
  - You can also connect script debugging tools directly to Unity to test your code more effectively.

# Unity Build Settings

- Now let us take a look at Android build settings and iOS build settings.
- For this lesson, please download the **Top-Down Shooter** game project from Chapter 6 and add it to Unity Hub as a new project.
  1. Download and the **Top-Down Shooter** ZIP file and extract it.
  2. Open **Unity Hub** ➜ Select **Projects** from the left-hand menu ➜ Click **Add** | **Add project from disk** in the top-right corner ➜ Select the downloaded and extracted project folder ➜ Click **Open**.

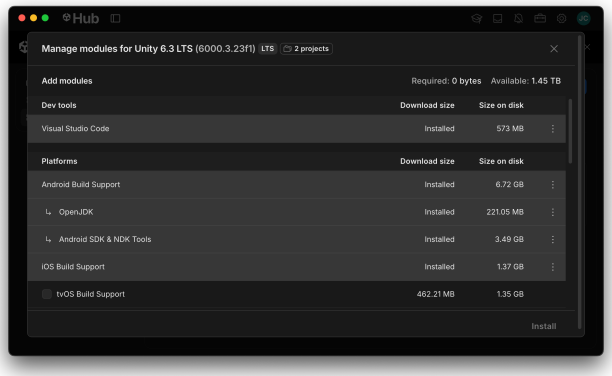


# Unity Build Settings



# Android Build Settings

- In Chapter 4, we discussed how to install the necessary tools to build Android apps using Unity.
- Android's development environment allows Unity to use the Software Development Kit (SDK) and the Native Development Kit (NDK) to build the apps directly within the Unity Editor.

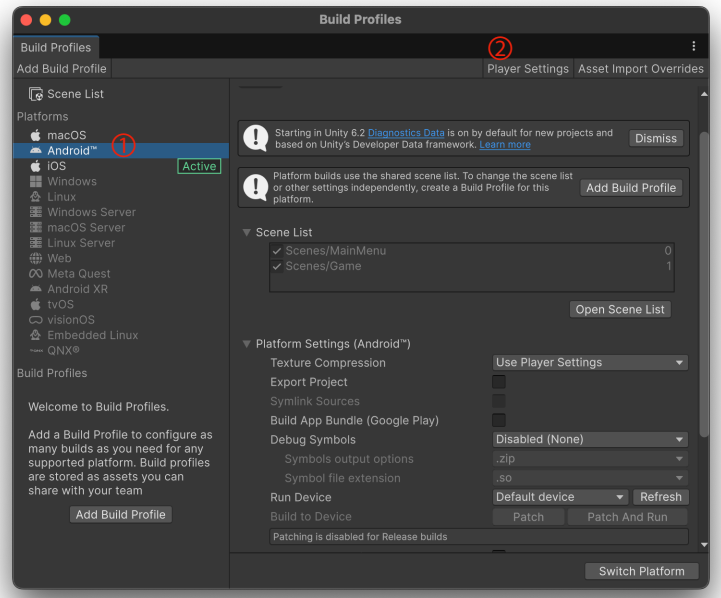


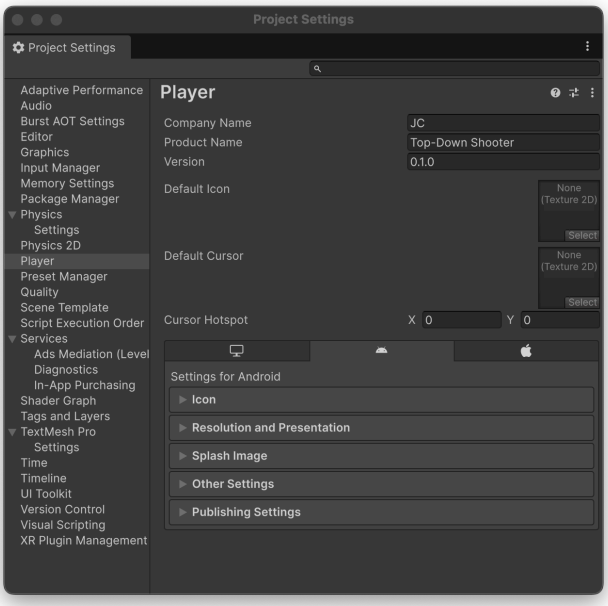



# Android Build Settings

- Go to **File** | **Build Profiles**.
- Select **Android** and click **Player Settings** tab.
  - You can also open this window by going to **Edit** | **Project Settings** and choosing the **Player** item in the left sidebar.
  - The player settings consist of shared settings across all the platforms, such as the name of the app, the version, the developer's name, and the icon.

# Android Build Settings






# Android Build Settings

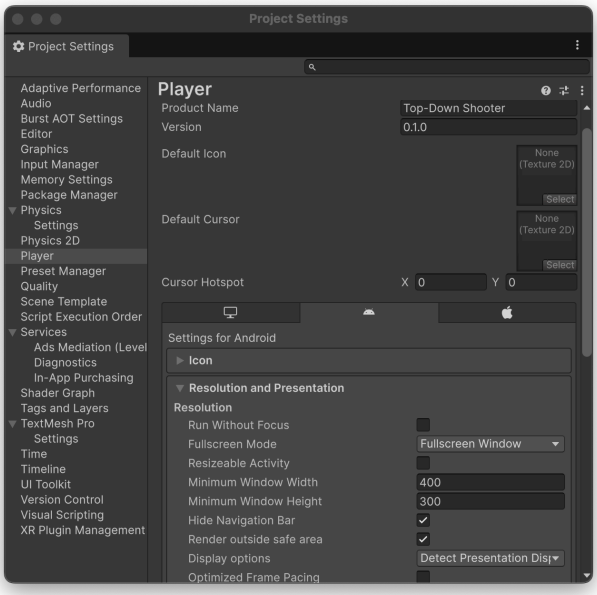
- Click the **Android** (🤖) tab and check the settings for Android.
  - **Icon**
    - This is used if you need different icons for different devices with various screen sizes.
    - You can upload different icons that match several resolutions.

# Android Build Settings

- **Resolution and Presentation**
  - This defines how your screen should be when you build it on an Android device.
  - There are various options to enable and disable full-screen window mode, hide the navigation bar, and forced aspect ratios.
  - You can lock this to just one orientation or allow **Auto Rotation** and tick only **Portrait** and **Portrait Upside Down** options.



# Android Build Settings



# Android Build Settings

- **Splash Image**
  - Unity has a default splash with its logo.

  - You can remove it if you have a paid Unity license.
  - We can leave it on default settings for now.



# Android Build Settings

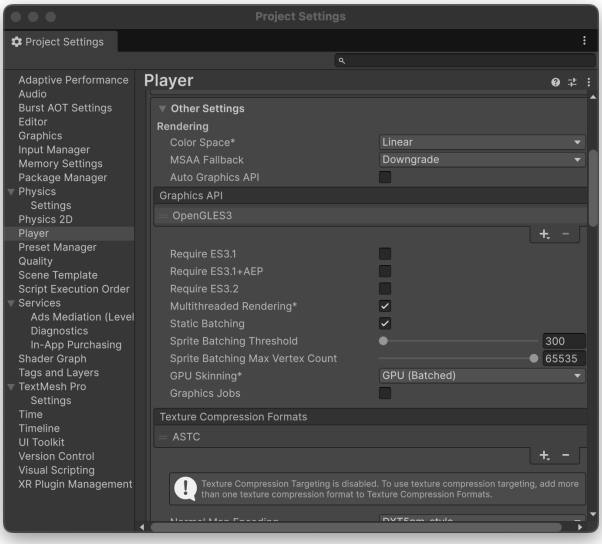
- **Other Settings**
  - This is the most important section when it comes to building AR applications.
  - This section will point out the necessary settings required to build AR apps for Android.
  - Choose the following for the options under **Other Settings**:
    - **Rendering** | **Color Space**: Set this option to **Linear**.

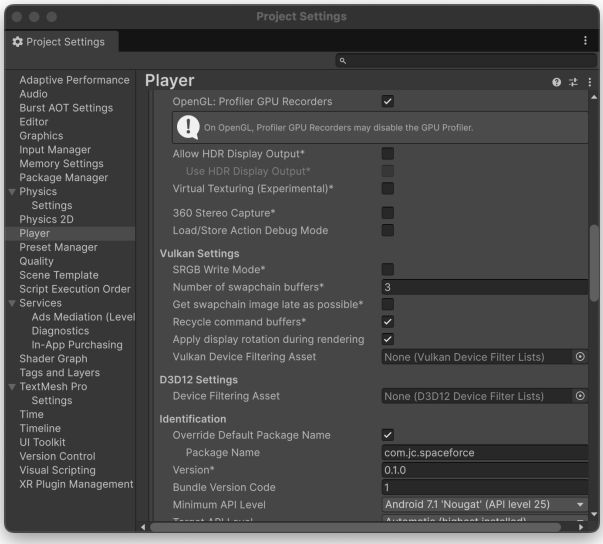
# Android Build Settings

- **Other Settings**
  - **Rendering** | **Graphics APIs**: Currently Android-based AR SDKs only support **OpenGLES3** API.
  - **Identification** | **Package Name**: This is the unique package name for your project. When you upload it to any app store, it is going to use this to uniquely identify you app. Usually, it is recommended to use **com.companyname.appnickname**.



# Android Build Settings



# Android Build Settings

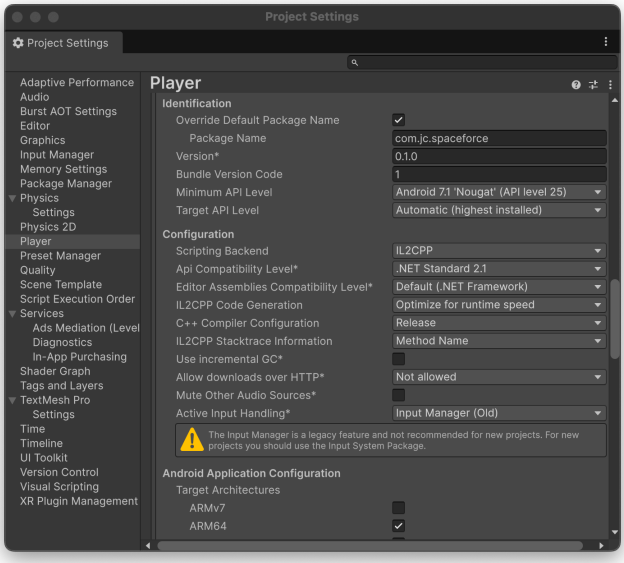
- **Other Settings**
  - **Identification** | **Versions**: When preparing a new version of the app, you can update the version number here. This value is used to identify the app version.
  - **Identification** | **Minimum API Level**: Android operating systems are updated regularly, and each version has its own API level. Currently, the minimum API level required to build an AR app is **Android 7.1 'Nougat' (API level 25)**.

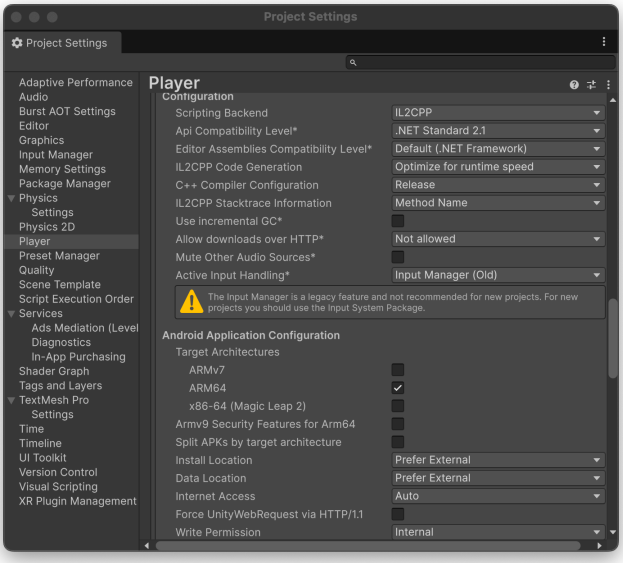


# Android Build Settings

- **Other Settings**
  - **Configuration** | **Scripting Backend**: Many recent Android devices now support 64-bit architecture. To support the 64-bit devices, the application must be compiled using a backend that supports 64-bit architecture. Therefore, set the **Scripting Backend** to **IL2CPP**.
    - If the app does not require any AR capabilities, we can use **Mono** and **ARMv7** as the scripting backend and the target architecture.
  - **Configuration** | **Target Architectures**: When you set the **Scripting Backend** to **IL2CPP**, you will get the option to select a target architecture. Select **ARM64** only, as AR apps will not work with any other architecture.

# Android Build Settings






# Android Build Settings

- **Publishing Settings**
  - Android uses a **Keystore** to sign an app during the build process.
  - App sining uses a certificate associated with the developer's singing key. This certificate helps identify the developer or app publisher.
  - App stores use the signing certificate to verify the app's identity and ensure that updates come from the same signing source.
  - Once an app is signed with a certificate, all future updates must be signed with the same certificate or a compatible sining key.

# Android Build Settings

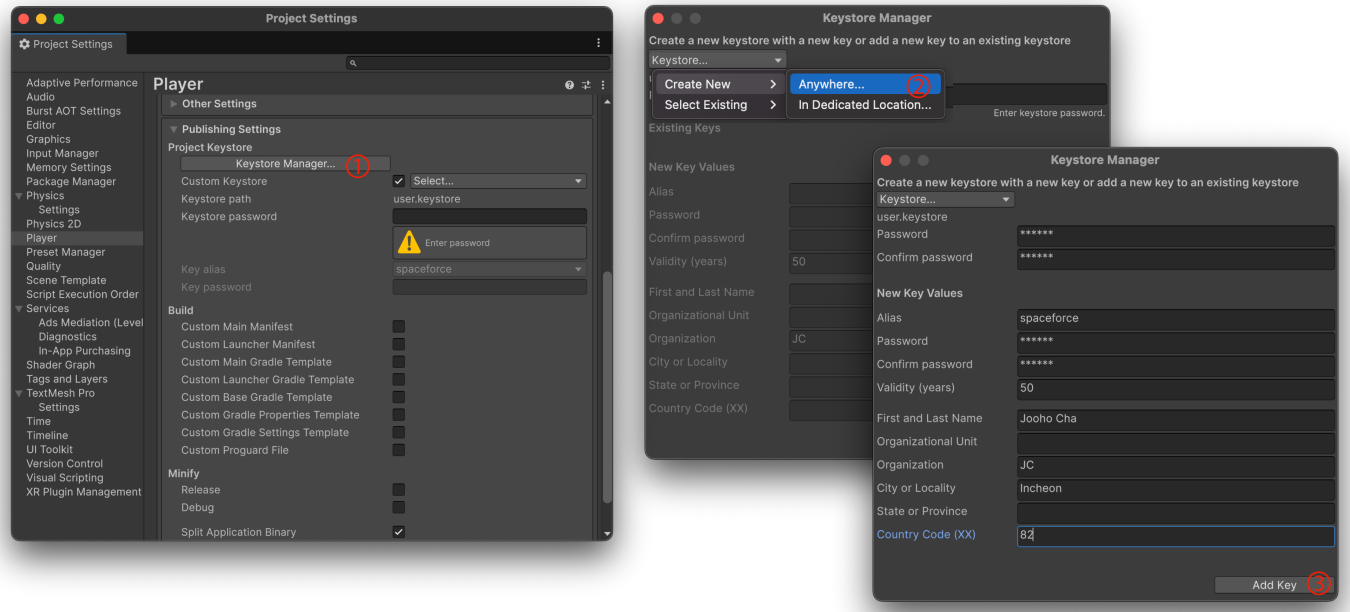
- **Publishing Settings**
  - You can leave these settings as they are if you are only testing the app on your phone. Unity uses a debug key and a certificate that are already available on your computer.
  - You need to create a new key if you want to publish your app on an app store.
  - For Android, Unity provides built-in tools that allow you to create your own Keystore for sining an app.



# Android Build Settings

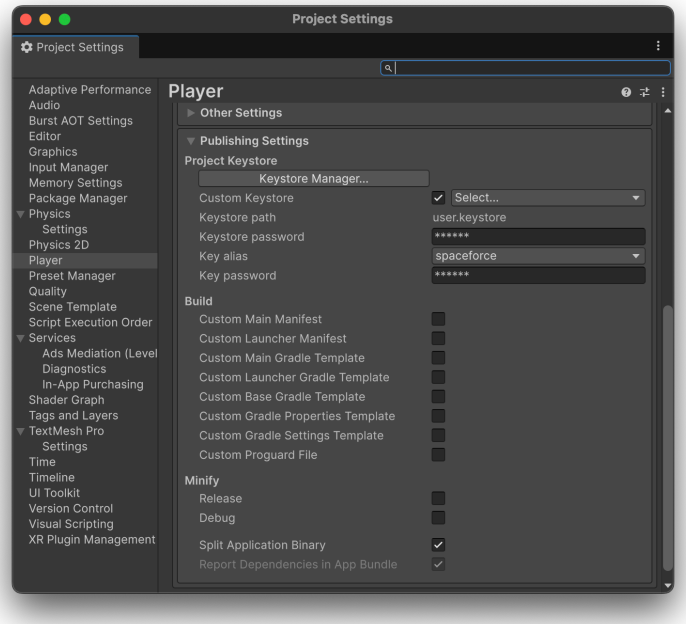
- **Publishing Settings**
  - If you plan to publish your app on an app store, click the **Keystore Manager** button to open the Keystore Manager window.
  - Here, you can load an existing Keystore file or create a new one.
  - It is recommended that you use a separate key for each application within a Keystore and store it securely in a safe location.

# Android Build Settings






# Android Build Settings



# Android Build Settings

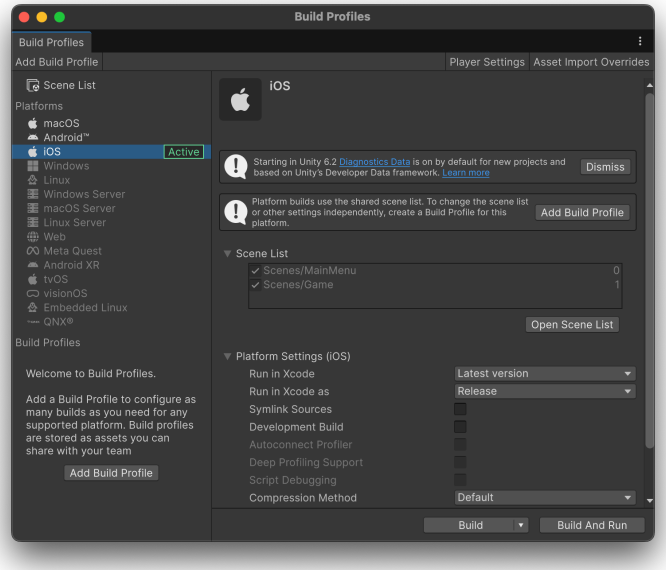
- **Publishing Settings**
  - Once the Keystore is loaded, you can select the loaded key under the **Custom Keystore** section.
  - Unity manages the application's manifest, so you can leave these settings at their default values for now.
  - If your app is large, you can enable **Split Application Binary** to split the app into multiple parts.
    - This allows app stores to distribute the app in smaller packages instead of one large file.

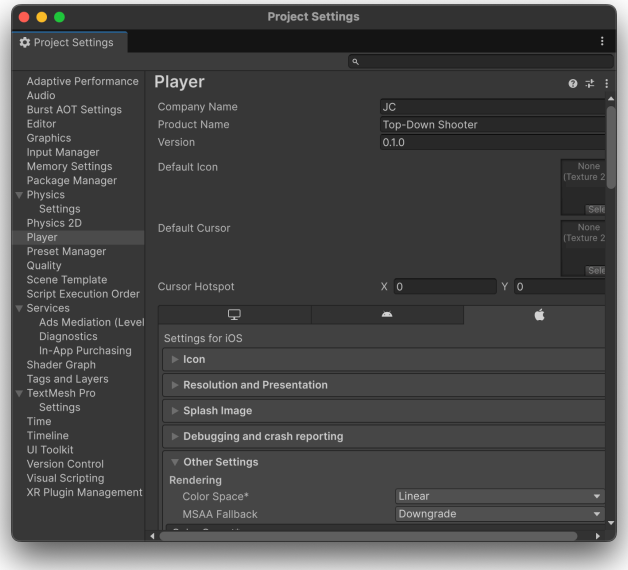


# iOS Build Settings

- Apple does not provide an SDK that allows Unity to build the final iOS app directly.
- Instead, Unity generates an **Xcode project**, which you can use to build an deploy the app to iOS devices.
- The settings for icons, resolution, and splash screens are similar to those for Android.
- Therefore, let's look at the **Other Settings** section under the **Apple ( )** tab in **Player Settings**.

# iOS Build Settings






# iOS Build Settings

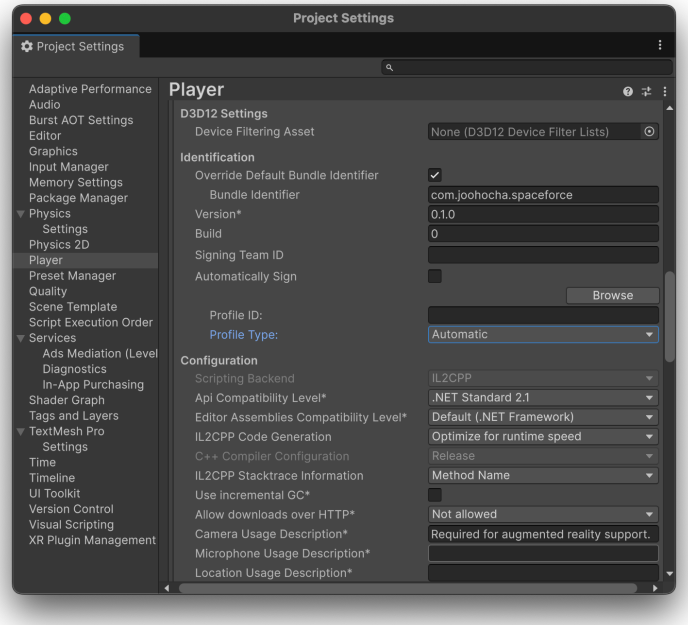
- **Identification | Bundle Identifier**
  - Set a unique identifier for your app.
  - You can use the same identifier across different platforms.
- **Identification | Signing Team ID**
  - Enter your Apple Developer Team ID.
  - This identifies your Apple Developer account.

# iOS Build Settings

- **Identification | Profile ID**
  - Specify the Provisioning Profile ID when using manual signing.
  - You can leave it blank when using automatic signing.
- **Identification | Profile Type**
  - Select the appropriate provisioning profile type.
  - Choose **Development** for testing or **Distribution** for publishing.



# iOS Build Settings



# iOS Build Settings

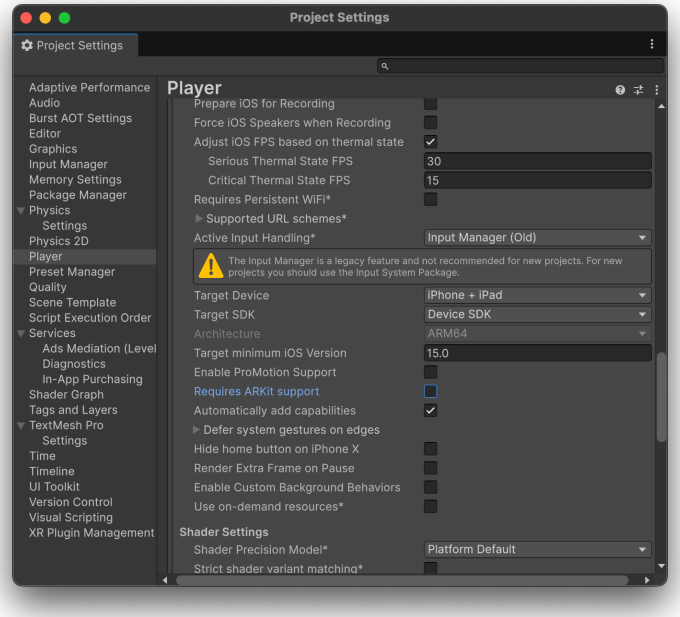
- **Configuration | Camera Usage Description**
  - AR apps require a camera, so enter a message explaining why the app needs access to the camera.
  - For example: "**Required for augmented reality support**".
- **Configuration | Target Device, SDK and minimum iOS Version**
  - Set the minimum iOS version to **15.0** or later.
  - Select **iPhone + iPad** to support both devices.
  - Select **Device SDK** to build the app for physical iOS devices.



# iOS Build Settings

- **Configuration | Requires ARKit support**
  - The Top-Down Shooter project does not require **ARKit**.
  - However, in the AR Foundation section, we should enable **Requires ARKit suppor**t to ensure that the app runs only on ARKit-compatible devices.
- These are the main settings we can configure in Unity.
- After Unity generates the Xcode project, we can make additional changes in Xcode.

# iOS Build Settings






# Building Unity Apps

- Let's go through the process of building an app and installing it on your devices.
  - Android and iOS use different build processes.
  - Let's start with Android.

# Building for Android

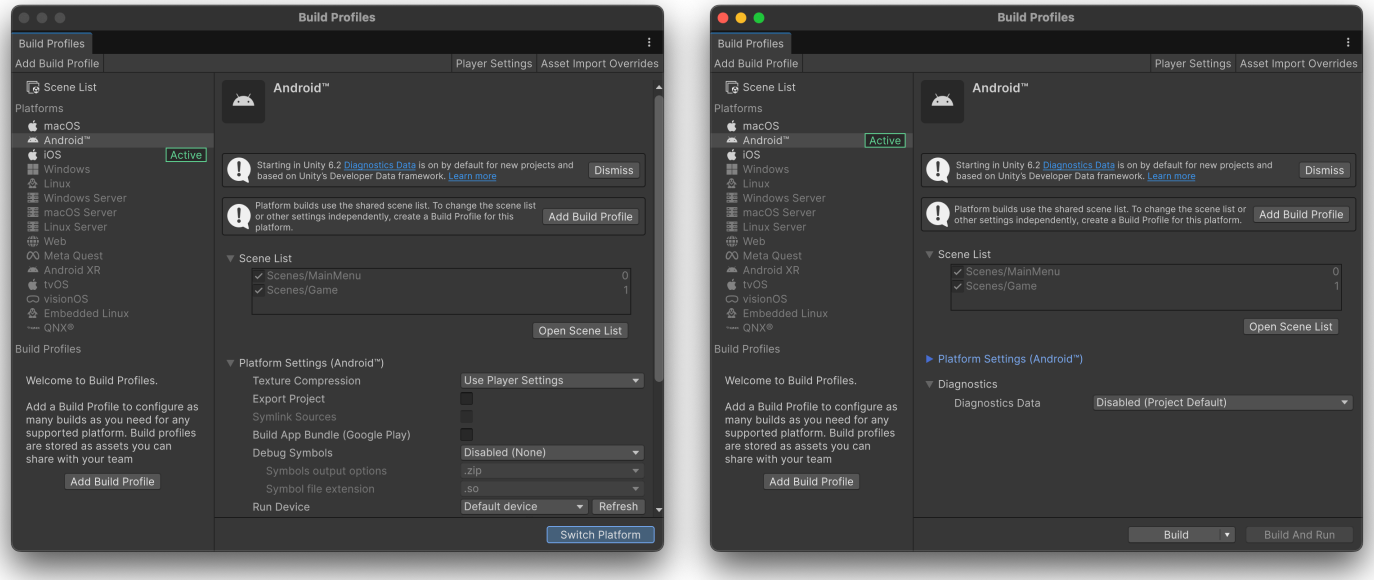
- When you build an Android app, Unity generates an Android Package (APK) file that you can install on your Android device.
- You can also use the **Build and Run**, which uses the Android Device Bridge (ADB) to connect to your device via USB or Wi-Fi and automatically install and launch the app.
- To understand both options, let's try building the app in both ways.



# Building for Android without ADB

- Open **Build Profiles** (**File** | **Build Profiles**).
- Select **Platform** as **Android** and click **Switch Platform** button.
- Check that required Scenes are listed and enabled in the **Scene List**, and click **Build** button.
- Select a folder to save the APK file, enter a file name (e.g. **build.apk**), and click **Save**.

# Building for Android without ADB






B B

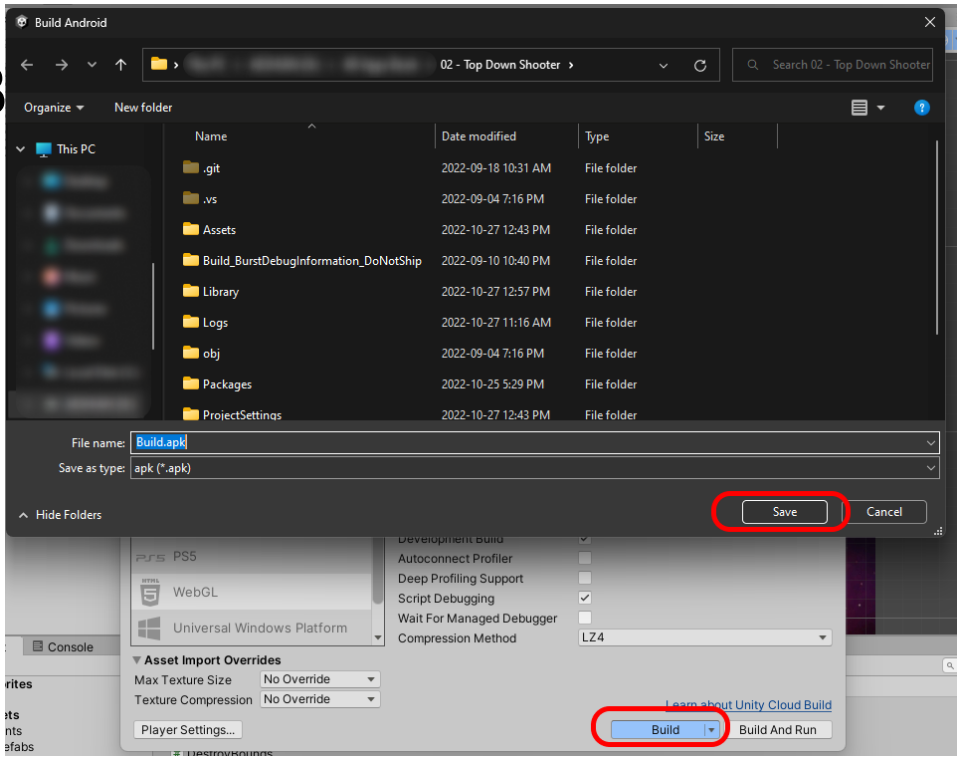



*Figure 7.4: Saving an Android build APK*

# Building for Android without ADB

- Once the build process is completed, you will have a **build.apk** file.
- Connect your Android device and copy the APK file to it.
- Open the APK file using a file manager to start the installation.
- Android blocks app installations from unknown sources by default. You may need to allow your file manager to **Install unknown apps**.
- Figure 7.5 shows how to enable this setting and the messages diesplayed when you open the **build.apk**.



# Building for Android without ADB

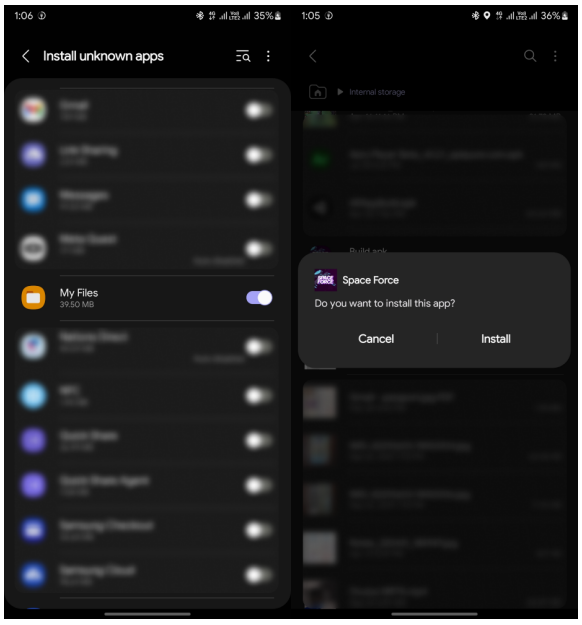



*Figure 7.5: The messages that appears when you tap the **build.apk** in the file manager*

# Building for Android with ADB

- This process can be time-consuming when testing the app repeatedly on your device.
- A quicker way is to build and install the app directly on your device using **Build and Run**.
  - To do this, you need to enable **Developer options** on your Android device.
  - Follow the steps below. The steps may vary slightly depending on your device manufacturer and model.



# Building for Android with ADB

1. On your Android device, open **Settings** and go to **About Phone**.
2. Find **Build number**. It may be under **software information**.
3. Tap **Build number** 5, 6 times until you see a message saying **Developer mode has been enabled**.
4. Go back to **Settings**, open **Developer options**, and enable **USB debugging**.
5. Connect your device to your computer using a USB cable. When prompted, allow **USB debugging**.

# Building for Android with ADB

6. In Unity, open **Build Profiles** and refresh the device list. Select your device from the **Run Device** drop-down.
7. Click **Build And Run**, choose a location and file name for the APK file, and click **Save**.
8. Unity will install and launch the app directly on your device, ready for testing. See Figure 7.6

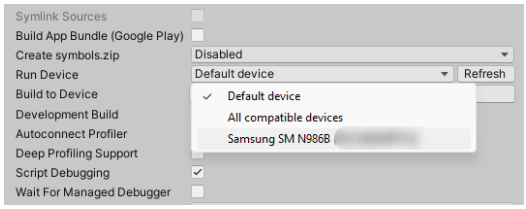
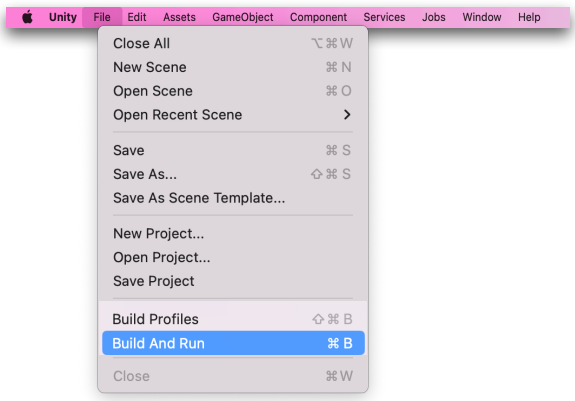



*Figure 7.6: Choosing the connected Android device to **Build And Run***



# Building for Android with ADB

- If you are testing your app repeatedly, you can select **Build and Run** directly from the **File** menu without opening the **Build Profiles**.



# Building for iOS

- Building an iOS app requires a Mac with Xcode installed.

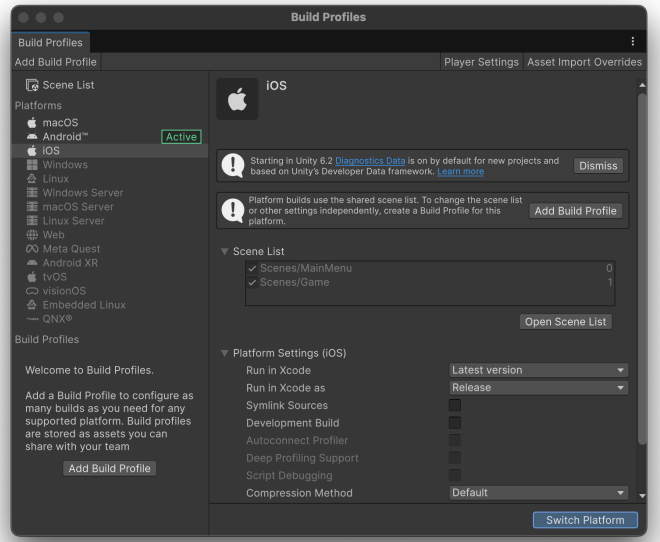
1. Open **Build Profiles** and select **iOS** as the **Platform**. Click **Switch Platform**. Unity may take some time to switch platforms.

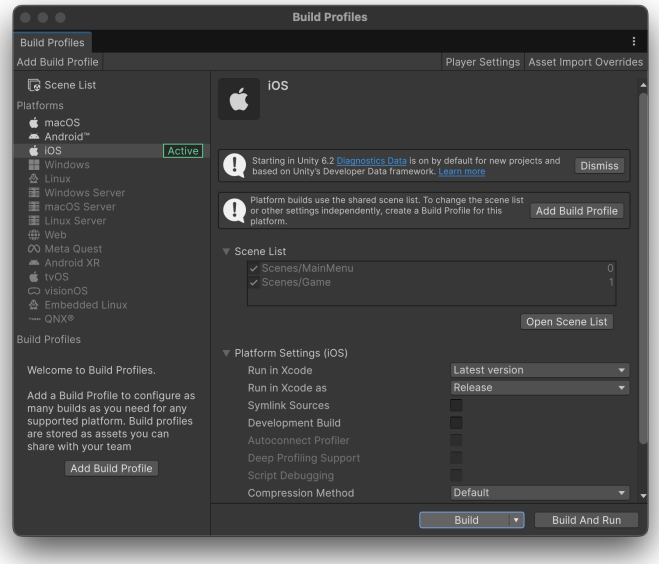
2. Click **Build** button to generate an Xcode project. Select or create a folder (e.g. **Xcode**) to save the project.

    - You can also select **Build and Run**, but it is recommended to build the project first and configure the singing setting in Xcode.

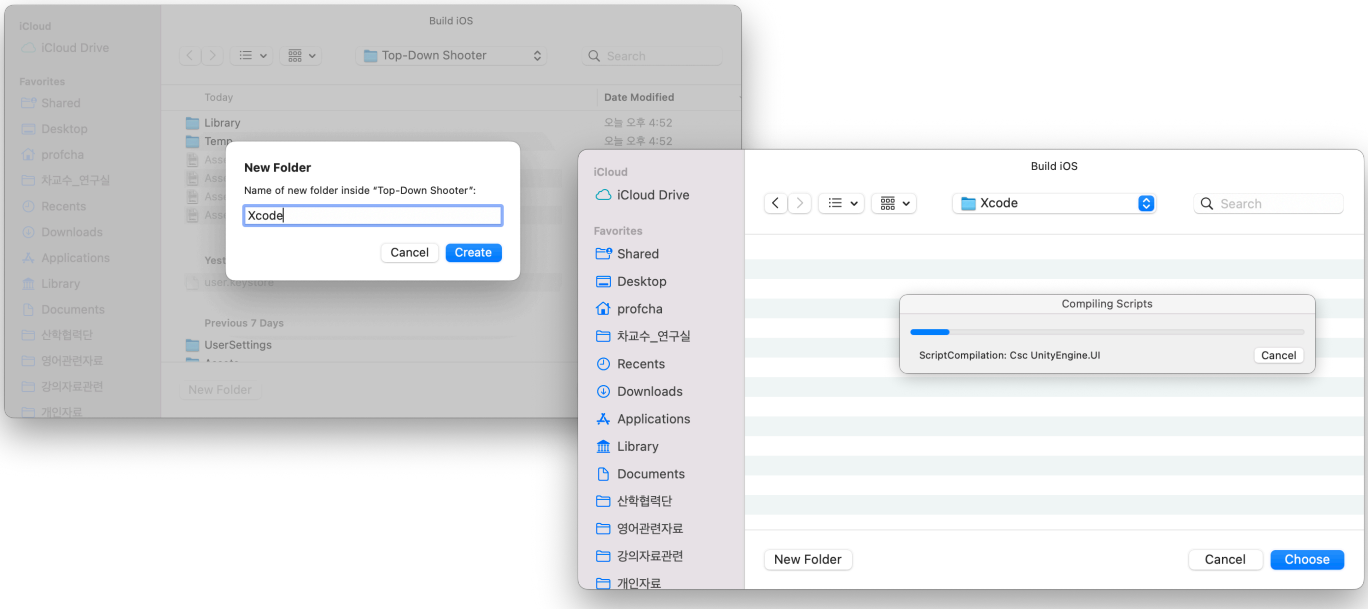


# Building for iOS



# Building for iOS

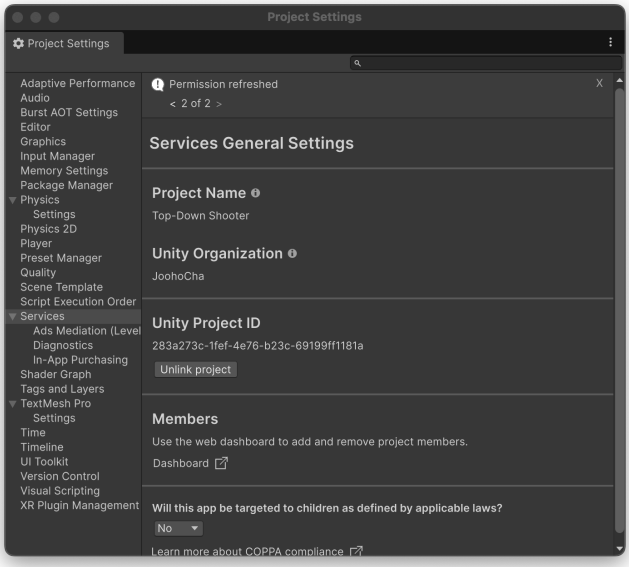





# Building for iOS

- If the **Missing Project ID** error occurs when building an iOS app in Unity 6, it may be because your Unity project is not linked to a Unity Cloud project.
- To resolve this issue, follow these steps:
  A. Open **Edit | Project Settings | Services**.
  B. Sign in with your Unity account.
  C. Select an **Organization**.
  D. Link your Unity project to an existing Cloud project or create a new one.
  E. Check your account permissions and project connection.
  F. Try building the iOS project again.

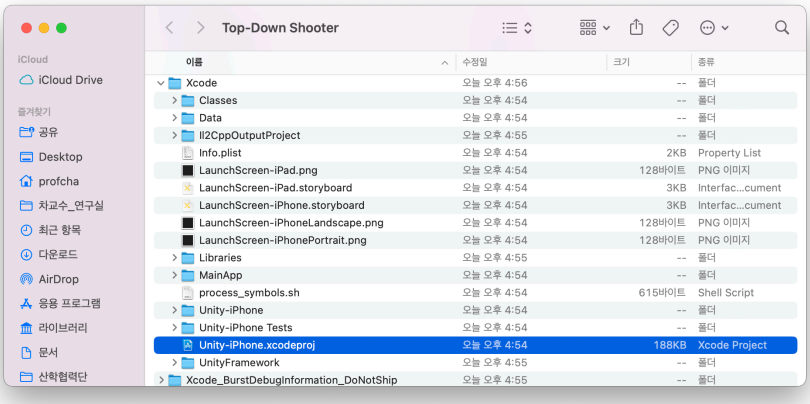
# Building for iOS






# Building for iOS

3. Once the build process is completed, open the build folder and duble-click **Unity-iPhone.xcodeproj** to open the generated Xcode project.



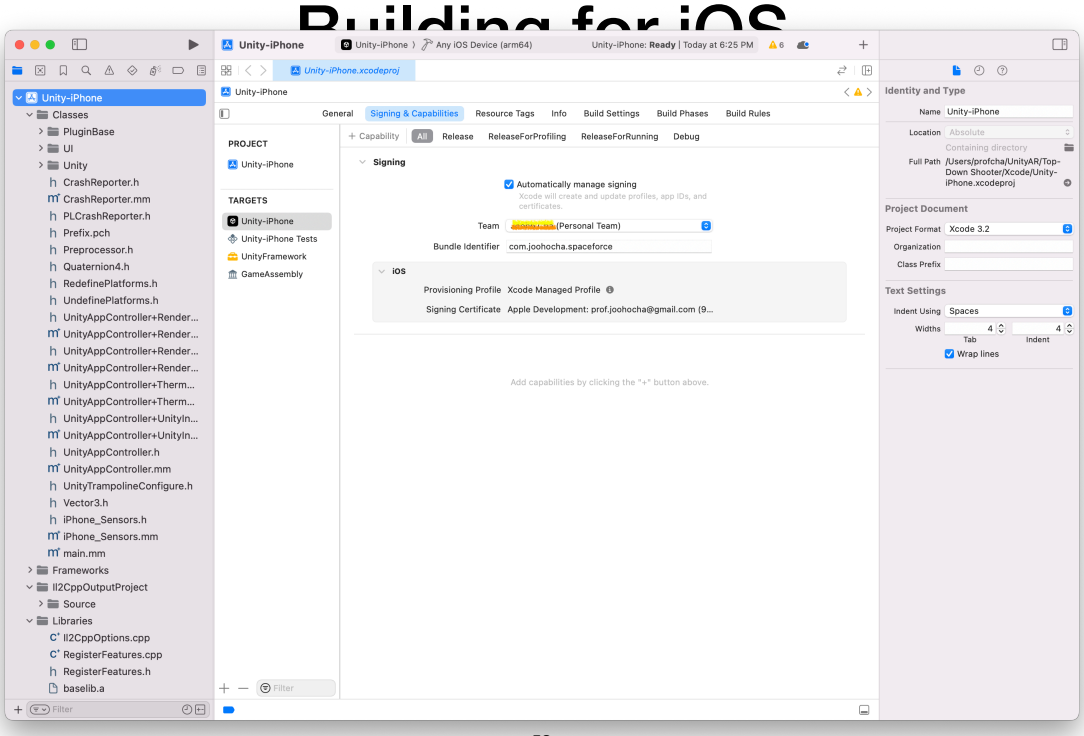
# Building for iOS

4. In the left sidebar of the Xcode project, select **Unity-iPhone**. Open the **Signing & Capabilities** tab and select **All** from the filter options.

5. Enable **Automatically manage signing** option. If you are signed in to Xcode with your Apple account, select your account and **Personal Team** from the **Team** drop-down menu. Make sure the **Bundle Identifier** is correct.



Building for iOS

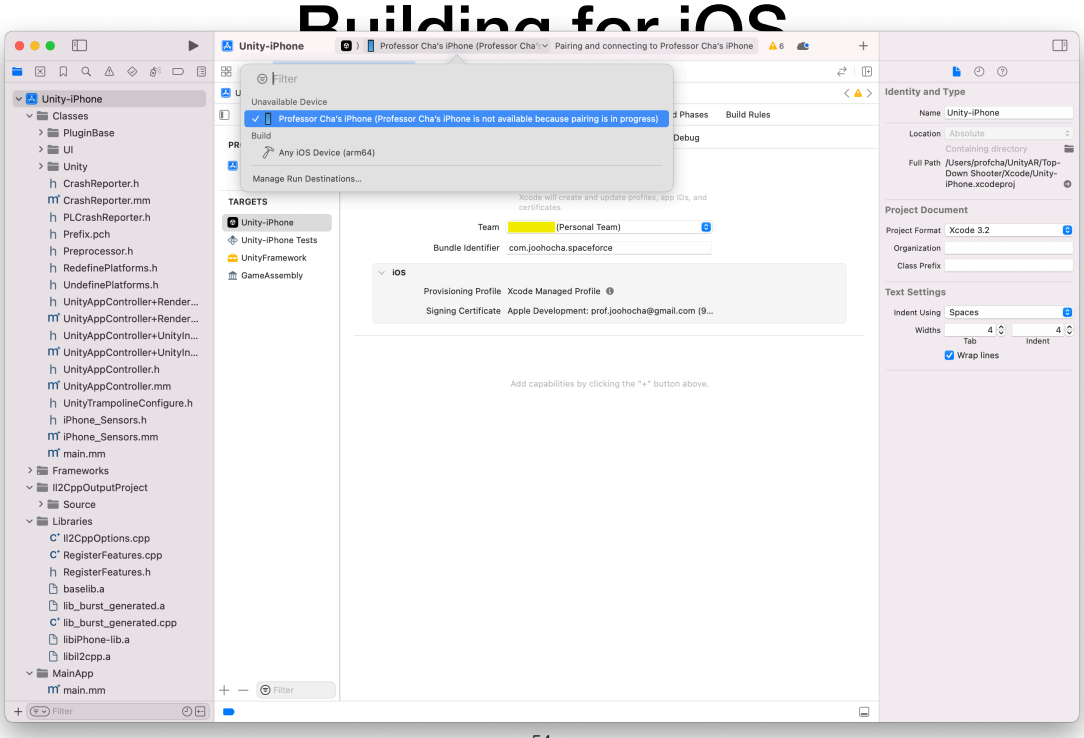


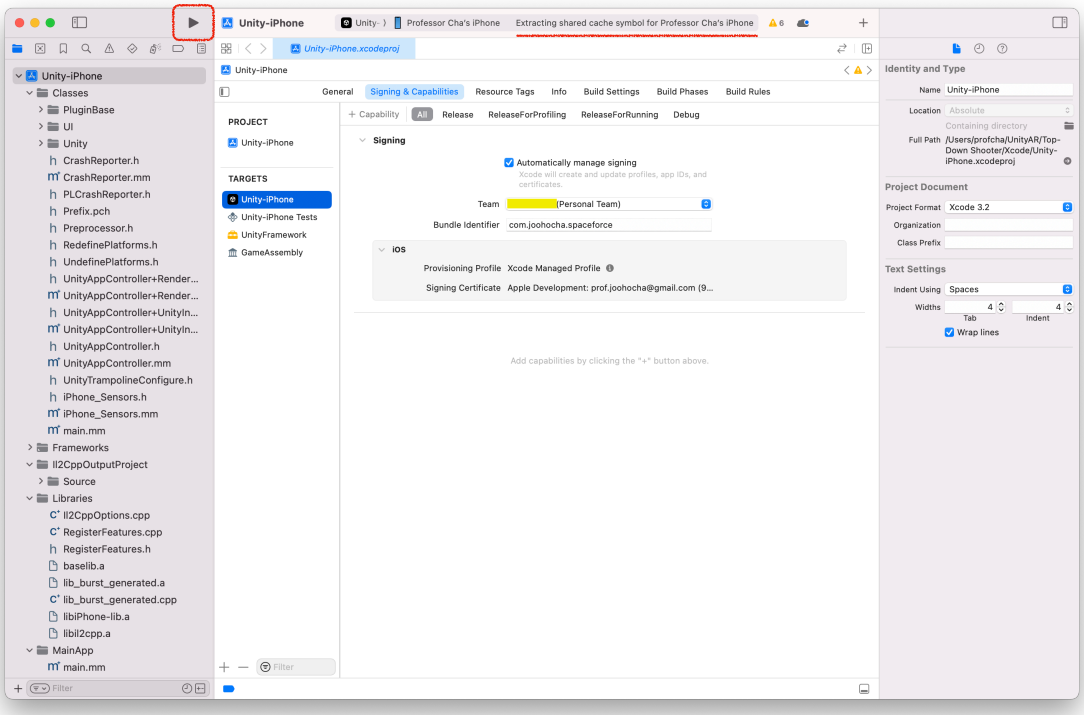
# Building for iOS

6. The first time you enable automatic signing, Xcode will generate a certificate for you. Now, connect your iOS device to your Mac using a USB cable.
7. Select your device from the device selector in the top toolbar.
8. Click the **Play (▶)** button to build and launch the app on your device.



Building for iOS

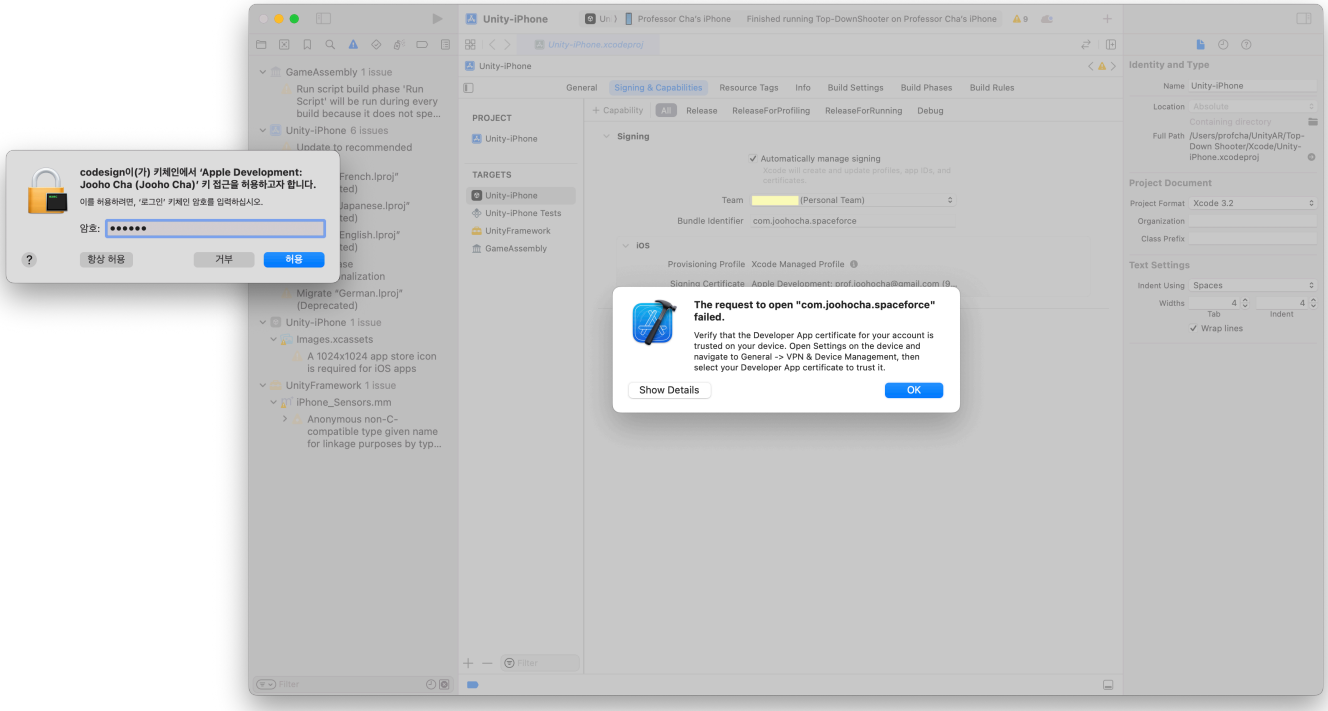






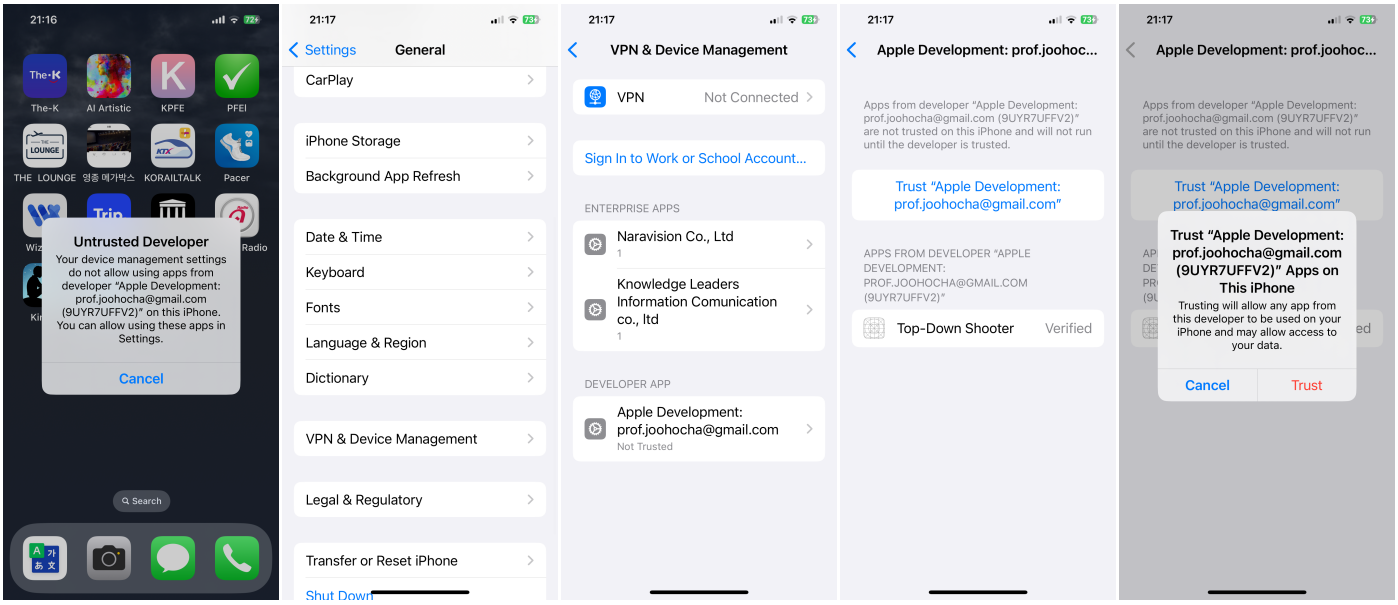

# Building for iOS

- This will start a secondary build process and generate an IPA file, which will be installed on your iOS device.
- During the second build, macOS may ask you to enter your keychain password. On your iPhone, you may also need to complete some security-related settings.
- Once the app is running, you can view the debug logs directly in Xcode.

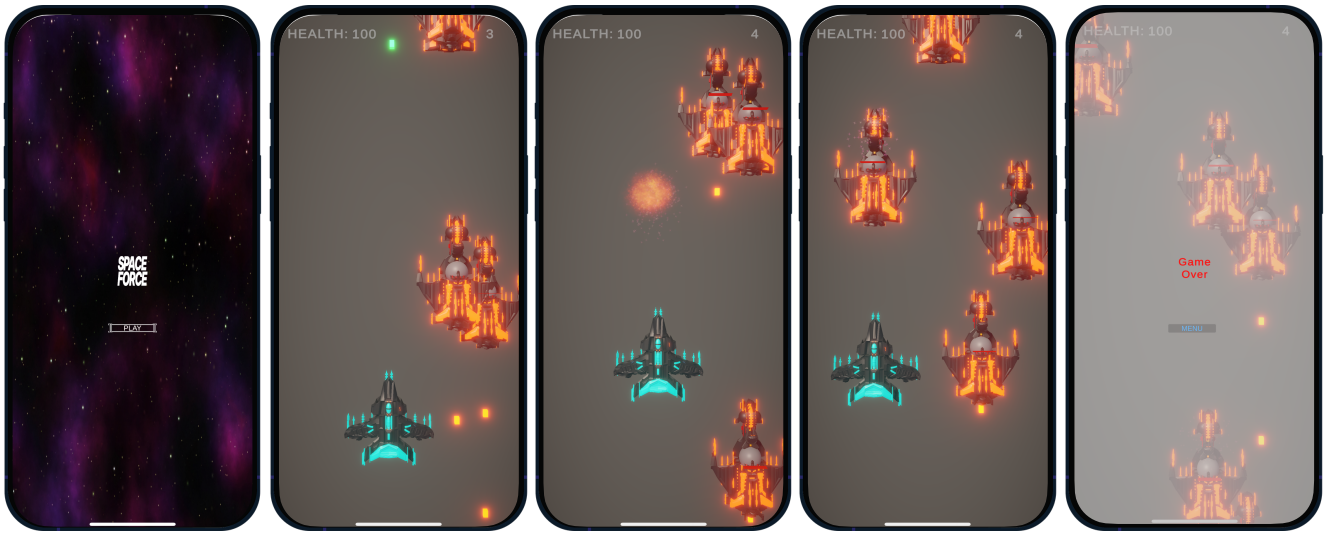





# Building for iOS



# Building for iOS






# Unity script debugging with Visual Studio

- Let us take a step back and look at how to properly debug your codes with Unity and Visual Studio.
- Script debugging allows us to check for issues in our codes and the status of the variables properly.
- This can be done in development time. Follow the given steps:

  1. For this exercise, let us use the **GameManger** script and check the **AppendScore(int)** method.

# Unity script debugging with Visual Studio

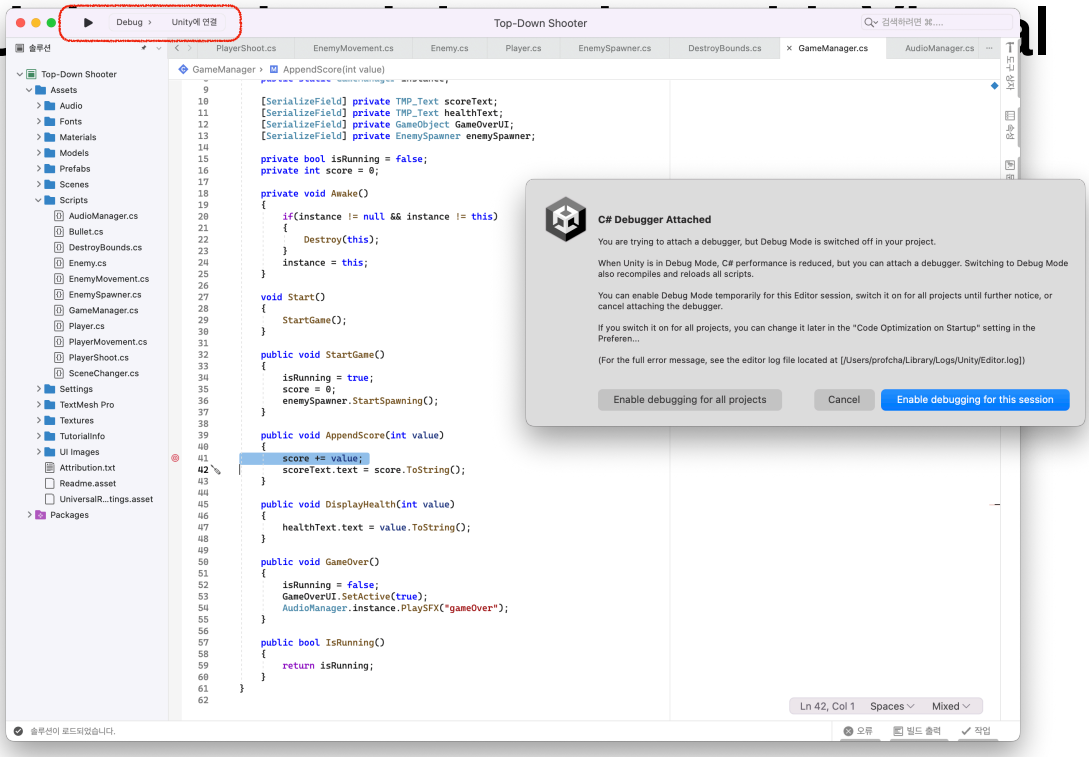
2. Open the **GameManger** script.
3. In order to check and debug a certain line of code, we need to create a **breakpoint**. Click on the leftmost column on the **socre += value** line of **AppendScore** method until you see a circle, and it will make a **breakpoint**. Now, when the game is running, and it reaches this point, the game will pause, and you will be able to analyze the code and variable values in that line.



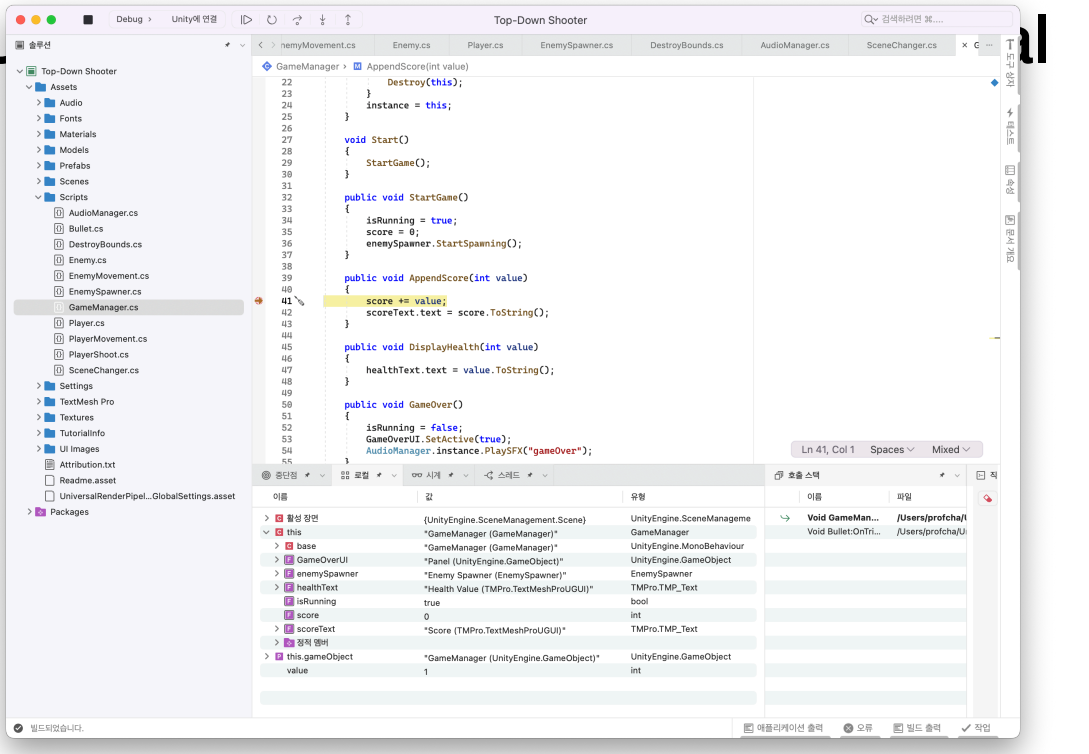
# Unity script debugging with Visual Studio

4. Once you make the **breakpoint**, select **Attach to Unity** from the top panel. This will initiate debugging and connect to Unity (in macOS, you need to click the play button in Visual Studio). Now go back to Unity and click the **Play** button.

- Now when you take a score, the game will pause and will launch Visual Studio with the current status of the game at that point.
- You will be able to see all the components attached to the game object and the values of each variable.


Debug
Unity에 연결
Top-Down Shooter
C# Debugger Attached
You are trying to attach a debugger, but Debug Mode is switched off in your project.
When Unity is in Debug Mode, C# performance is reduced, but you can attach a debugger. Switching to Debug Mode also recompiles and reloads all scripts.
You can enable Debug Mode temporarily for this Editor session, switch it on for all projects until further notice, or cancel attaching the debugger.
If you switch it on for all projects, you can change it later in the "Code Optimization on Startup" setting in the Preferen...
(For the full error message, see the editor log file located at [/Users/profcha/Library/Logs/Unity/Editor.log])
Enable debugging for all projects
Cancel
Enable debugging for this session




Debug
Unity에 연결
Top-Down Shooter

# Debugging the app for both platforms

- Let us go through some of the tools that are available for both iOS and Android platforms to view the console log while the app is running on the device.
- **Using the ADB console for Android**
  - Unity comes bundled with the Android SDK, which includes the Android device bridge tool to connect and push data into the device.
  - We can use the same to open the console log, also known as the logcat, which logs every process on the device.

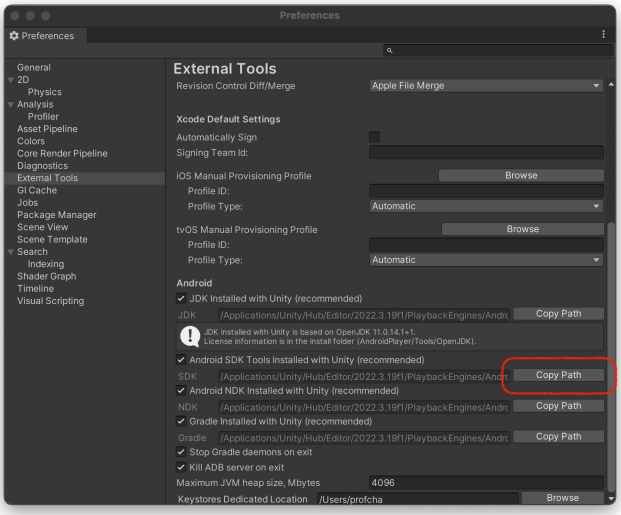


# Debugging the app for both platforms

- **Using the ADB console for Android**
  - We can filter the detail log using specific parameters to show only the items we want in our app. Follow these steps:
    1. Connect your Android device running the Unity app.
    2. In Unity, go to **Edit** | **Preferences** (in MacOS, it is **Unity** | **Settings...**). Select **External Tools** and scroll down to the **Android SDK Tools** path. Click **Copy Path** button.

# Debugging the app for both platforms

- **Using the ADB console for Android**






# Debugging the app for both platforms

- **Using the ADB console for Android**

3. Open the Windows terminal or MacOS terminal and open the path you copied by this command:

   **cd <<The path you copied>>\platform-tools**

4. Now type the following:

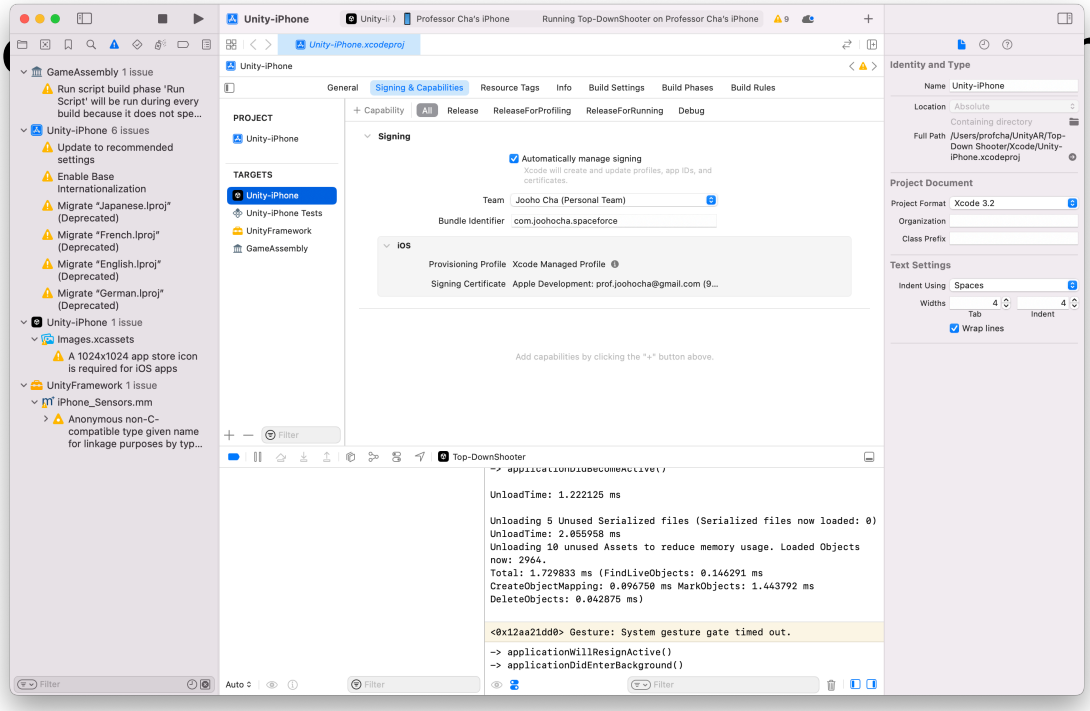
   **./ adb logcat -s Unity ActivityManager PackageManager dalvikvm DEBUG**

# Debugging the app for both platforms

- **Using the Xcode console for iOS**
  - Xcode provides a cleaner log-checking system by immediately opening the console within Xcode as soon as it starts running on the device.
  - When you connect your device and click the play button to build the app into the device, you can see the filtered console log on Xcode until you click the stop button.



# D ns



# Conclusion

- We discussed the basics of building and testing the application on a mobile phone.
- We also went through basics of debugging and explored logging from the Unity editor.



# Key points

- There are different build settings requirements for different operating systems.
- Unity used the Android SDK directly to build and install the app on a mobile phone.
- Unity exports an Xcode project which can be later customized, signed, and deployed on an iOS device.
- Visual Studio debugger can be connected with Unity to test and debug the code at runtime.
- Console log can be viewed using the ADB logcat and Xcode runtime console.

# Q & A